

Quantum Repeaters

Quantum Repeaters: Concept and Architecture

The Concept of a Quantum Repeater

In classical optical networks, signal attenuation over long distances is mitigated using optical amplifiers. However, in quantum communication, the **no-cloning theorem** dictates that an unknown quantum state (qubit) cannot be copied or amplified without destroying the encoded quantum information. Consequently, photon loss, which accumulates exponentially with increasing channel length, makes direct long-distance transmission of quantum states over optical fibers infeasible.

To circumvent this limitation, quantum communication systems employ **Quantum Repeaters**. Instead of attempting to prevent photon loss entirely, this approach accepts its inevitability and utilizes distributed entangled pairs to transfer quantum states through quantum teleportation and a process known as **entanglement swapping**.

A memory-based quantum repeater network divides the total communication distance into shorter, manageable segments. Entanglement is distributed across these relatively short segments. The repeaters contain **quantum memories** that temporarily store intermediate quantum bits (ancilla qubits) while adjacent segments successfully generate their own entanglement. Once adjacent links share entanglement, a joint Bell-state measurement is performed at the intermediate repeater node. This entanglement swapping destroys the intermediate qubits but projects the outer nodes into an entangled state, effectively extending the range of entanglement without directly transmitting a single photon over the entire distance.

Quantum Repeater Architecture in the MDP Environment

To optimize the operations of these repeaters—such as determining the exact moment to store, swap, or discard qubits due to environmental decoherence—the network is modeled as a **Markov Decision Process (MDP)**.

Network Topology

The simulated network is structured as an n -segment repeater chain comprising $n + 1$ nodes, sequentially labeled from 0 to n . The primary objective of the Deep Reinforcement Learning (DRL) agent is to successfully distribute an entangled pair of qubits between the terminal nodes (node 0 and node n). The remaining $n - 1$ intermediate nodes serve as stations where the entanglement swapping operations can be executed.

State Representation

The state space of the quantum repeater chain is represented by an $(n+1) \times (n+1)$ matrix. This matrix tracks the existence and the specific age (in time steps) of the entanglement links between any two nodes. In the specific Gymnasium environment adaptation, a placeholder value (such as 1 or null) is utilized to indicate the absence of a connection between a given pair of nodes.

Action Space

At each time step, the agent can perform two distinct types of actions to manage the entanglement links:

- **Entanglement Swapping:** Represented by a one-dimensional array of $n - 1$ elements. It dictates which intermediate nodes should attempt to perform a Bell-state measurement to connect their left and right adjacent entangled links.
- **Discarding Links:** Represented by a two-dimensional boolean upper triangular matrix, which is flattened into a 1D array. If an entry is set to true (or 1), the corresponding aged entanglement link is purposefully destroyed. This frees up the quantum memory and prevents severe decoherence from ruining the overall fidelity of the network.

System Transitions and Dynamics

During a single time step in the MDP, the architecture undergoes the following transitions based on the underlying laws of quantum physics and the agent’s actions:

- **Entanglement Generation:** Each elementary segment independently attempts to generate an entangled photon pair with a base success probability of p . A successful attempt creates a new entanglement link with an initial age of 0.
- **Discarding Links:** Any link explicitly marked for discarding by the agent is eliminated, and its corresponding state matrix entry is reset.
- **Entanglement Swapping:** If an intermediate node j connects to node i and node k , and a swap action is executed, a new extended link is formed between i and k . Crucially, the age of this newly formed link is the sum of the ages of the two constituent links ($\text{state}[i][k] = \text{state}[i][j] + \text{state}[j][k]$), while the original shorter links are reset.

Reward Mechanism

The DRL agent is motivated by a reward formulation based on the Secret Key Rate (SKR) of the Entanglement-based BB84 protocol. The environment applies small negative penalties (-0.5) at each step to encourage rapid entanglement distribution. When the end-to-end link between node 0 and node n is successfully established, the agent receives a significantly scaled positive reward proportional to the generated secret key fraction. This heavily penalizes older end-to-end links, reflecting the exponential decay of fidelity modeled by the quantum memory coherence time τ_c .

Mathematical Modeling of Quantum Repeaters and DRL Optimization

1. Quantum Network and Decoherence Modeling

The fundamental modeling of quantum noise over time relies on the Pauli channel framework. Instead of deterministic error propagation, the noise is treated as a probabilistic mixture of Pauli operators. The state evolution under a Pauli channel is given by:

$$N(\rho) = \sum_{j=0}^3 a_j P_j \rho P_j^\dagger$$

where $P_j \in \{I, X, Z, Y\}$ represents the Pauli matrices, and a_j denotes the probability of each error occurring.

A critical property of this formulation is that dephasing noise (a Pauli Z error) accumulates additively. Therefore, when an entanglement swapping operation connects two adjacent segments, the age of the new continuous link is strictly additive:

$$\text{state}[i][k] = \text{state}[i][j] + \text{state}[j][k]$$

During the network's initial generation phase, physical photon loss in optical fibers is mathematically characterized by transmissivity. The probability of successfully generating an initial two-qubit entangled state in a segment during one time step is:

$$p = p_x \cdot \eta$$

where p_x is the photon creation efficiency and η is the overall transmissivity of the optical fiber.

2. Reward Formulation (Secret Key Rate)

The MDP environment's overarching goal is to maximize the final Secret Key Rate (SKR). The base efficiency of the quantum key distribution is quantified as:

$$R = Y \cdot r$$

where Y is the raw key rate (yield) and $r \in [0, 1]$ is the secret key fraction, representing the portion of bits that remain provably secure after post-processing procedures.

For the simulated entanglement-based BB84 protocol, the specific reward calculation models the exponential decay of fidelity within the quantum memories. The positive reward obtained upon establishing an end-to-end link is the SKR defined as:

$$R_{BB84} = Y \cdot \left(1 - h \left\{ \frac{1}{2} \left[1 - e^{-\frac{t}{\tau_c}} \right] \right\} \right)$$

where:

- t is the total accumulated age (in time steps) of the resulting end-to-end entanglement link.
- τ_c is the coherence time of the utilized quantum memories.
- $h\{x\}$ is the binary entropy function, defined as $-x \log_2 x - (1-x) \log_2 (1-x)$.

3. Deep Reinforcement Learning (DRL) Formulation

The optimization of operations is handled by a Proximal Policy Optimization (PPO) agent operating within an Actor-Critic framework. The agent’s stochastic policy is parameterized by the weights of a neural network θ , yielding $\pi_\theta(a|s)$, which outputs the probability of taking an action given a state.

For a sampled trajectory τ , the accumulated discounted reward across the horizon is:

$$R_0 = \sum_{t=0}^T \gamma^t r_t$$

where $\gamma \in [0, 1]$ is the discount factor prioritizing earlier rewards. The agent maximizes the expected return via targeted policy gradient updates:

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} [R_0] \Big|_{\theta=\theta_k}$$

To reduce variance during training and enhance stability, an advantage function $A(s, a)$ replaces the raw expected return. This evaluates an action’s performance relative to a baseline expected state-value. The state-value $V^\pi(s_t)$ and action-value $Q^\pi(s_t, a_t)$ functions are formally defined as:

$$V^\pi(s_t) = \mathbb{E}_{a_t, s_{t+1}, \dots \sim \pi_{\theta}} [R_t]$$

$$Q^\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}, a_{t+1}, \dots \sim \pi_{\theta}} [R_t]$$

The Advantage is mathematically their difference:

$$A(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

Because calculating the exact action-value function is computationally intractable for the large state space of a multi-node repeater network, the critic neural network approximates the state-value $V^\pi(s_t)$. The specific advantage estimate at time t used to guide the PPO actor is evaluated using the final trajectory’s BB84 reward:

$$A_t = R_{BB84}(\tau_L) - V^\pi(s_t)$$

This formulation directly evaluates actions based on how much they improve the final secret key rate over the current policy’s average baseline.

Deep Reinforcement Learning (DRL) Formulation: A Step-by-Step Breakdown

1. The Agent and the Policy Function

In Deep Reinforcement Learning, the **agent** is the decision-maker interacting with an environment. The core of this agent is its **policy**, denoted as π .

The formulation begins by defining a **stochastic policy** parameterized by a neural network: $\pi_\theta(a|s)$.

- π (**Pi**): Represents the policy itself—the strategy the agent uses to decide what to do.
- θ (**Theta**): Represents the weights of the neural network. If you are building this in a framework like PyTorch, θ represents all the trainable parameters (weights and biases) of your `nn.Module`. By updating θ , you change the policy.
- $(a|s)$: Read as “action a given state s .” Because it is a stochastic (probabilistic) policy, the network does not output a single fixed action. Instead, it outputs a probability distribution over all possible actions for that specific state.

2. The Accumulated Discounted Reward (R_0)

The goal of RL is not just to get immediate rewards, but to maximize the total reward over time. For a sampled **trajectory** τ (a full sequence of states and actions from start to finish), the accumulated discounted reward is defined as:

$$R_0 = \sum_{t=0}^T \gamma^t r_t$$

Breaking down the terms:

- R_0 : The total return from time step $t = 0$ to the end of the trajectory.
- T : The time horizon, or the final time step of the episode.
- r_t : The scalar reward received at a specific time step t .
- γ (**Gamma**): The discount factor, where $\gamma \in [0, 1]$. This is crucial. If $\gamma = 1$, all future rewards are valued equally to immediate rewards. If $\gamma = 0$, the agent only cares about the immediate next reward. Usually set around 0.99, it ensures that the agent prioritizes rewards obtained sooner rather than later, and helps the mathematical sum converge.

3. Policy Gradient Updates

To improve the policy, we need to adjust the neural network weights θ so that trajectories with higher rewards become more probable. This is done via **gradient ascent** (since we want to maximize the return):

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} [R_0] \Big|_{\theta=\theta_k}$$

Breaking down the terms:

- θ_k and θ_{k+1} : The network weights at the current step k and the next updated step $k + 1$.
- α (**Alpha**): The learning rate (step size). It controls how large of an update we make to the weights.
- ∇_{θ} (**Nabla**): The gradient operator with respect to the network weights. It gives us the direction of steepest ascent to increase our objective.

- $\mathbb{E}_{\tau \sim \pi_\theta}[R_0]$: The expected (average) return over multiple trajectories τ sampled by following the current policy π_θ .

4. Value Functions and the Actor-Critic Framework

Directly updating based on the raw return R_0 can lead to high variance (noisy updates) during training. To fix this, **Actor-Critic** architectures introduce “baseline” evaluations using Value Functions.

State-Value Function (V^π)

$$V^\pi(s_t) = \mathbb{E}_{a_t, s_{t+1}, \dots \sim \pi_\theta}[R_t]$$

The state-value function $V^\pi(s_t)$ acts as the **Critic**. It estimates how good it is to simply *be* in state s_t . It returns the expected total future reward if the agent starts from s_t and follows the policy π until the end.

Action-Value Function (Q^π)

$$Q^\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}, a_{t+1}, \dots \sim \pi_\theta}[R_t]$$

The action-value function (Q-function) evaluates how good it is to take a *specific action* a_t in state s_t , and *then* follow the policy π thereafter.

5. The Advantage Function (A)

The core concept of PPO and Actor-Critic is the **Advantage function**:

$$A(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

- **The Intuition:** The Advantage asks, “How much better was taking action a_t compared to the average action I would normally take in state s_t ?”
- If $A > 0$: The action was better than average. The gradient update should increase the probability of this action.
- If $A < 0$: The action was worse than the baseline. The gradient update should decrease the probability of this action.

6. Practical Approximation for the Quantum Network

In complex environments like multi-node repeater networks, the state and action spaces are too massive to calculate the exact $Q^\pi(s_t, a_t)$. Therefore, the researchers replaced the theoretical Q-function with an empirical observation:

$$A_t = R_{BB84}(\tau_L) - V^\pi(s_t)$$

- $R_{BB84}(\tau_L)$: This serves as a real-world stand-in for the theoretical Q -value. It is the actual final secret key rate achieved at the end of the episode (trajectory length τ_L).

- $V^\pi(s_t)$: The critic neural network’s baseline estimate of what the reward *should* have been from state s_t .

By subtracting the predicted baseline V^π from the actual achieved reward R_{BB84} , the algorithm directly evaluates whether the sequence of actions starting at time t resulted in a better or worse final key rate than normally expected.

Detailed Breakdown: Practical Approximation for the Quantum Network

To understand this section, we need to bridge the gap between theoretical reinforcement learning and the physical reality of programming a simulation. The core issue here is dealing with the limitations of computing power when facing complex environments.

1. The Problem: The Curse of Dimensionality

In a theoretical Actor-Critic framework, the advantage is calculated using the action-value function:

$$A(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

The term $Q^\pi(s_t, a_t)$ asks a mathematically demanding question: *If I am in state s_t , and I take action a_t , what is the exact mathematically expected average reward of every single possible future trajectory from this point forward?*

In the quantum repeater environment, calculating this is **intractable** (computationally impossible).

- **Massive State Space:** The state is an $(n+1) \times (n+1)$ matrix tracking the existence and age of every possible entanglement link.
- **Massive Action Space:** At every time step, the agent can choose multiple combinations of intermediate nodes for entanglement swapping, while simultaneously choosing which aged links to discard using a boolean upper triangular matrix.
- **Stochasticity:** Quantum entanglement generation is probabilistic (it succeeds with probability p). Furthermore, quantum memory decoherence adds randomness.

Because there are billions of possible ways the episode could play out after taking action a_t , calculating the exact theoretical average Q -value is impossible.

2. The Solution: Empirical Observation (Monte Carlo Return)

Since the agent cannot calculate every possible future, it does the next best thing: it just plays out *one* specific future and sees what actually happens. This is what the text means by replacing the theoretical Q -function with an **empirical observation**.

Instead of asking “*What is the expected average reward?*”, the algorithm asks “*What was the actual reward I got at the end of this specific run?*”

3. Deconstructing the Formula: $A_t = R_{BB84}(\tau_L) - V^\pi(s_t)$

This formula is the practical, code-level implementation of the Advantage function for this specific quantum network problem. Let us look at the terms individually.

The Stand-In for Q: $R_{BB84}(\tau_L)$

In this environment, the agent only receives a meaningful positive reward at the very end of the episode (at time τ_L), when it successfully connects node 0 to node n . This final reward is the Secret Key Rate, denoted as R_{BB84} .

Because the intermediate actions (like swapping or discarding at time t) do not give immediate positive rewards, we evaluate how good an action a_t was by looking at the final score of the episode it belonged to. Therefore, $R_{BB84}(\tau_L)$ acts as a stand-in for our Q -value. It represents the actual observed return of the trajectory.

The Baseline: $V^\pi(s_t)$

While we cannot calculate the exact theoretical average of all futures, we can train a neural network (the Critic) to guess it. $V^\pi(s_t)$ is the Critic network’s baseline estimate. It looks at the current quantum network state s_t (the ages of all current links) and predicts: *“Based on past training, if the board looks like this, I expect our final Secret Key Rate to be X .”*

The Resulting Advantage: A_t

By subtracting the Critic’s prediction from the actual final result, we determine the **Advantage**. This tells the algorithm how to update the policy network (the Actor):

- **If $A_t > 0$:** The actual final key rate R_{BB84} was higher than the Critic’s baseline prediction V^π . This means the actions taken starting from time t were unusually good. The algorithm will mathematically tweak the Actor network’s weights to make those actions more likely in the future.
- **If $A_t < 0$:** The actual final key rate was lower than expected. The actions taken from time t led to a poor outcome (perhaps the agent waited too long and the qubits decohered). The algorithm will adjust the weights to make those actions less likely.

Conclusion

In summary, because the quantum environment is too vast to calculate exact probabilities, the researchers use the actual outcome of the episode (R_{BB84}) to judge the quality of the actions, comparing it against a neural network’s educated guess (V^π) to determine if the agent performed better or worse than normal.

Why R_{BB84} Replaces the Q-Function

To answer your question directly: **Yes, exactly.** The value R_{BB84} is the actual secret key rate achieved because the agent was in state s_t , executed action a_t , and subsequently followed policy π until the episode finished.

To understand why a single final outcome is a mathematically valid replacement for the theoretical Q -value, we need to look at a core concept in reinforcement learning called **Monte Carlo Estimation**.

1. Expectation vs. Sampling

In the theoretical Actor-Critic formulation, the true action-value function is an **expectation** (a probability-weighted average):

$$Q^\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}, a_{t+1}, \dots \sim \pi_\theta}[R_t]$$

This equation mathematically means: *“Run an infinite number of simulations starting from state s_t , take action a_t , follow policy π to the end of time, and average all their final scores.”*

Since we cannot run infinite simulations from every single state during training, we rely on a statistical technique called sampling.

2. The Monte Carlo Sample

During training, the agent plays out episodes (trajectories). When the agent reaches time step t :

1. It observes the state s_t .
2. It samples an action a_t based on its current policy network π .
3. From time $t + 1$ onward, it continues to use policy π to pick actions until the end of the episode (time τ_L).

At the end of this specific run, the environment calculates the final Secret Key Rate, $R_{BB84}(\tau_L)$.

Because this entire trajectory was generated by following policy π after taking action a_t , the final reward $R_{BB84}(\tau_L)$ acts as a single, valid, real-world sample of the theoretical $Q^\pi(s_t, a_t)$.

3. The Problem of Variance (The “Lucky/Unlucky” Run)

You might immediately spot a flaw in this logic: quantum entanglement is probabilistic. What if the agent made a brilliant move at time t , but got extremely unlucky with photon loss later in the episode, resulting in a terrible R_{BB84} ?

If we use this single “unlucky” sample as our Q -value, the calculated Advantage ($A_t = R_{BB84} - V^\pi$) will be negative, and the neural network will penalize action a_t , even though it was the mathematically correct choice!

This is known as the problem of **High Variance**. A single sample is unbiased, but highly volatile.

4. How the Algorithm Fixes the Variance

The Deep Reinforcement Learning algorithm overcomes this volatility through two mechanisms:

- **The Law of Large Numbers:** The agent does not train on just one episode. The paper notes that the agent is trained over 3×10^7 time steps. Over millions of iterations, the agent will visit similar states and take similar actions thousands of times. The “lucky” and “unlucky” probabilistic outcomes eventually cancel each other out. The neural network’s gradient updates naturally average these single samples out, approximating the true expectation $\mathbb{E}$.
- **The Critic Network (V^π):** This is exactly why the Critic network is so important. By subtracting $V^\pi(s_t)$ from R_{BB84} , the algorithm establishes a stabilizing baseline. If the agent gets a massive reward, but the Critic already predicted a massive reward based on the excellent state s_t , the Advantage is near zero. The policy only updates strongly when the result is significantly better or worse than the learned average.

Summary

You have understood it perfectly. We replace the computationally impossible average (Q^π) with a single, actual observed result (R_{BB84}) generated by following the policy. While one sample is noisy, repeating this process over millions of training steps allows the neural network to mathematically converge on the true expected value of the actions.

Understanding Time Steps in the MDP Simulation

In reinforcement learning, and specifically within this quantum repeater simulation, a **time step** serves as the fundamental, discrete unit of time. You can think of it as one “turn” or one “tick of the clock” in the environment.

What Happens in a Single Time Step?

In the context of the modeled quantum network, one time step represents the exact physical duration required for the network to complete a full round of basic operations between adjacent nodes. Specifically, during a single time step, the following sequence occurs:

- **Entanglement Attempt:** The physical hardware attempts to generate an elementary entangled link between neighboring nodes.
- **Classical Communication:** The nodes exchange classical information to communicate whether that probabilistic entanglement generation attempt succeeded or failed.
- **Agent Action:** The DRL agent observes the updated state of the entire network (the link ages and matrices), decides on an action (such as performing entanglement

swapping at intermediate nodes or discarding aged links), and the environment updates accordingly.

The Scale of Training: 3×10^7 Time Steps

When the paper specifies that the agent is trained for 3×10^7 **time steps**, it means the agent interacts with the environment for 30 million individual decision-making moments.

Because a single episode terminates as soon as the agent successfully connects the terminal nodes (node 0 and node n), these 30 million steps are not a single continuous run. Instead, they are distributed across many thousands of separate training episodes. This massive scale gives the agent enough experience to encounter a wide variety of stochastic outcomes (both lucky successes and unlucky decoherence events). Over time, this allows the neural network to average out the variance and learn a highly optimized policy.

Results

QuantumRepeaterEnv without probabilistic entanglement swapping

PPO Agent

Mean reward per 10^5 steps: 987.82 ± 158.91

Mean successes per 10^5 steps: 34.66

Mean end-to-end link age: 5.03 ± 0.53

Memory-Cutoff Baseline

Mean reward per 10^5 steps: 4340.91 ± 163.81

Mean successes per 10^5 steps: 922.28

Mean end-to-end link age: 14.75 ± 3.36

QuantumRepeaterEnv with probabilistic entanglement swapping

50 Episodes, each with 10^5 time steps

PPO Agent

Mean reward per 10^5 steps: 45.29 ± 34.19

Mean successes per 10^5 steps: 2.44

Mean end-to-end link age: 7.61 ± 2.44

Memory-Cutoff Baseline

Mean reward per 10^5 steps: 546.35 ± 54.15

Mean successes per 10^5 steps: 118.88

Mean end-to-end link age: 14.84 ± 3.31

100 Episodes, each with 10^5 time steps

PPO Agent

Mean reward per 10^5 steps: 45.29 ± 34.19

Mean successes per 10^5 steps: 2.44

Mean end-to-end link age: 7.61 ± 2.44

Memory-Cutoff Baseline

Mean reward per 10^5 steps: 546.35 ± 54.15

Mean successes per 10^5 steps: 118.88

Mean end-to-end link age: 14.84 ± 3.31